

JAVA

Professional Study Notes

Beginner → Basic → Intermediate → Advanced

College Exams • Viva • Practical • Technical Interviews • DSA Preparation

Concept-first notes with examples, code walkthroughs, comparisons, revision and an important DSA question bank.

Contents

Java Fundamentals

Variables & Data Types

Input, Conditions & Loops

Methods & Recursion

Arrays & Strings

Classes, Objects & Constructors

OOP: Encapsulation, Inheritance & Polymorphism

Abstraction & Interfaces

Exception Handling

Collections Framework

Generics & Wrapper Classes

File I/O Basics

Multithreading Basics

Lambda & Stream Basics

JVM, JDK, JRE & Memory

equals(), hashCode(), final & Immutability

Debugging & Complexity

Final Revision + Important DSA Questions

Chapter 1: Java Fundamentals

Java is a high-level, class-based, object-oriented language. Source code is compiled into bytecode, and the JVM executes that bytecode. This design allows the same compiled program to run on systems with compatible JVMs.

Java execution flow

Source (.java) → javac compiler → Bytecode (.class) → JVM → Execution

First program

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

- public class Main → declares the class.
- main(...) → standard application entry point.
- System.out.println(...) → prints a line.

JDK vs JRE vs JVM

Term	Meaning	Purpose
JVM	Java Virtual Machine	Executes bytecode
JRE	JVM + runtime libraries	Runs Java programs
JDK	JRE + development tools	Develops, compiles and runs

■ Important: Java source normally uses .java and compiled bytecode normally uses .class.

Interview & Viva

- Why platform independent? → Bytecode is designed to be executed by platform-specific JVM implementations.
- What is bytecode? → Intermediate code produced by the Java compiler.

Chapter 2: Variables, Data Types & Operators

A variable is a named storage location associated with a type. Java has eight primitive types plus reference types.

Type	Example	Purpose
byte	byte x=10;	Small integer
short	short x=10;	Integer
int	int x=10;	Common integer
long	long x=10L;	Large integer
float	float x=2.5f;	Decimal
double	double x=2.5;	Decimal
char	char c='A';	Character
boolean	boolean b=true;	Logical value

Operators

- Arithmetic: + - * / %
- Relational: > < >= <= == !=
- Logical: && || !
- Assignment: = += -= *= /=
- Unary: ++ --

```
int a = 5;
int b = 2;
System.out.println(a / b); // 2
System.out.println(a % b); // 1
```

■■ 5/2 is 2 because both operands are integers. Integer division removes the fractional part.

Common mistakes

- Confusing == with content equality for objects.
- Forgetting L on long literals or f on float literals when required.
- Assuming all variables behave identically; primitives and references have different semantics.

Chapter 3: Input, Conditions & Loops

Programs use input, decisions and repetition to turn basic statements into useful algorithms.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
System.out.println("You entered: " + n);
```

if / else

```
if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

Loops

Loop	Use
for	Counter-controlled repetition and array traversal
while	Repeat while a condition remains true
do-while	Run the body at least once

Nested loops

A nested loop is a loop inside another loop. It is useful for matrices, patterns and pair comparisons. Two full n-sized nested loops commonly lead to $O(n^2)$ time.

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Common mistakes

- Off-by-one errors.
- Forgetting to update a while-loop variable.
- Accessing an array outside its valid index range.

Chapter 4: Methods & Recursion

A method is a reusable block of code that performs a task.

```
static int add(int a, int b) {  
    return a + b;  
}
```

```
public static void main(String[] args) {  
    int result = add(10, 20);  
    System.out.println(result);  
}
```

- Parameters are variables in the method definition.
- Arguments are actual values supplied at the call site.
- return sends a value back to the caller.

Recursion

Recursion means a method calls itself. A correct recursive method needs a base case and must make progress toward it.

```
static int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1);  
}
```

$\text{factorial}(3) \rightarrow 3 \times \text{factorial}(2) \rightarrow 3 \times 2 \times \text{factorial}(1) \rightarrow 3 \times 2 \times 1 \times \text{factorial}(0) \rightarrow 6.$

■■ Missing or unreachable base cases can cause `StackOverflowError`.

Chapter 5: Arrays & Strings

An array stores a fixed number of values. Indexing starts at 0 and the last valid index is length - 1.

```
int[] arr = {10, 20, 30, 40};

for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

2D array

```
int[][] m = {{1, 2}, {3, 4}};
System.out.println(m[1][0]); // 3
```

Strings

String represents a sequence of characters and is immutable. Methods such as substring() return String values rather than modifying the original String.

```
String s = "Java";
System.out.println(s.length()); // 4
System.out.println(s.charAt(0)); // J
System.out.println(s.substring(1)); // ava
System.out.println(s.equals("Java")); // true
```

■ Use equals() for String content comparison. == compares references for objects.

Array vs ArrayList

Array	ArrayList
Fixed size	Resizable
arr.length	list.size()
Can hold primitives directly	Uses object elements; primitives autobox

Chapter 6: Classes, Objects & Constructors

A class is a blueprint; an object is an instance of that class. Fields store state and methods define behavior.

```
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void show() {  
        System.out.println(name + " " + age);  
    }  
}
```

- Constructor has the same name as its class and no return type.
- `new Student(...)` creates an object and invokes its constructor.
- `this` refers to the current object.

Common mistake: **`void Student(...)`** is a method, not a constructor.

Chapter 7: OOP: Encapsulation, Inheritance & Polymorphism

Object-oriented programming organizes software around objects containing state and behavior.

Encapsulation

Encapsulation controls access to internal state and exposes safe operations.

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

Inheritance & overriding

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
```

Runtime polymorphism

```
Animal a = new Dog();
a.sound(); // Bark
```

Overloading	Overriding
Same name, different parameter list	Child provides compatible replacement for inherited method
Usually compile-time selection	Runtime dispatch for overridden instance methods
Multiple signatures	Same method signature/compatible return rules

■ Interview: Be able to explain why `Animal a = new Dog()` can call Dog's overridden `sound()`.

Chapter 8: Abstraction & Interfaces

Abstraction focuses on essential behavior while hiding unnecessary implementation details.

```
abstract class Vehicle {
    abstract void start();
    void stop() { System.out.println("Stop"); }
}

class Car extends Vehicle {
    @Override
    void start() { System.out.println("Car starts"); }
}
```

Interface

```
interface Payment {
    void pay();
}

class UPI implements Payment {
    public void pay() {
        System.out.println("Paid");
    }
}
```

Abstract class	Interface
Can contain instance state and implemented methods	Defines a contract; may also contain default/static methods
A class extends one class	A class can implement multiple interfaces

Chapter 9: Exception Handling

An exception is an abnormal condition that can interrupt normal execution. Java provides structured mechanisms to handle such conditions.

```
try {  
    int x = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
} finally {  
    System.out.println("Finished");  
}
```

Keyword	Role
try	Code that may throw
catch	Handles a matching exception
finally	Usually used for cleanup
throw	Explicitly throws an exception
throws	Declares possible exceptions in a method signature

Checked vs unchecked

Checked	Unchecked
Compiler requires handling/declaring many checked exceptions	Usually RuntimeException subclasses
Example: IOException	Examples: NullPointerException, ArithmeticException

Chapter 10: Collections Framework

Collections are reusable data structures and are essential for Java DSA.

Type	Idea	Common implementation
List	Ordered; duplicates allowed	ArrayList
Set	Unique elements	HashSet
Queue	Queue/priority processing	ArrayDeque, PriorityQueue
Map	Key-value pairs	HashMap, TreeMap

ArrayList

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
System.out.println(list.get(0));
System.out.println(list.size());
```

HashMap frequency counting

```
HashMap<Integer, Integer> freq = new HashMap<>();
for (int x : arr) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}
```

■ This pattern appears in many DSA problems: frequency, duplicate detection, counting and fast lookup.

Chapter 11: Generics, Wrapper Classes & Enums

Generics provide compile-time type safety for classes and methods.

```
ArrayList<String> names = new ArrayList<>();  
names.add("Anurag");  
names.add("Java");
```

Primitive	Wrapper
int	Integer
double	Double
char	Character
boolean	Boolean
long	Long

Autoboxing converts primitive → wrapper; unboxing converts wrapper → primitive.

```
Integer a = 10; // autoboxing  
int b = a;      // unboxing
```

Enum

```
enum Level { LOW, MEDIUM, HIGH }  
Level level = Level.HIGH;
```

Chapter 12: File I/O Basics

File I/O lets programs persist and retrieve information from files. Modern Java commonly uses Path and Files.

```
import java.nio.file.*;

Path path = Path.of("data.txt");
Files.writeString(path, "Hello Java");
String text = Files.readString(path);

System.out.println(text);
```

Real-world uses

- Reading datasets.
- Writing reports/configuration.
- Exporting application results.
- Working with logs and text files.

■■ File operations can fail because of paths, permissions or I/O errors; production code should handle relevant exceptions.

Chapter 13: Multithreading Basics

A thread is a path of execution within a process. Multiple threads can make progress concurrently.

```
class Task implements Runnable {  
    public void run() {  
        System.out.println("Task running");  
    }  
}
```

```
Thread t = new Thread(new Task());  
t.start();
```

start()	run()
Starts a new thread of execution	Direct call is an ordinary method call
Used to begin thread execution	Does not create a new thread by itself

Race condition

When threads access shared mutable state without proper coordination, timing can produce incorrect results. Synchronization and higher-level concurrency utilities can help.

Chapter 14: Lambda & Stream API Basics

A lambda is a concise way to represent behavior for a functional interface.

```
List<Integer> nums = Arrays.asList(1, 2, 3, 4);  
nums.forEach(x -> System.out.println(x));
```

Stream pipeline

```
List<Integer> result = nums.stream()  
    .filter(x -> x % 2 == 0)  
    .map(x -> x * 2)  
    .toList();
```

Step-by-step: filter keeps 2 and 4 → map changes them to 4 and 8 → toList creates [4, 8].

- filter → select elements.
- map → transform elements.
- sorted → order elements.
- forEach → consume elements.
- toList/collect → produce a result.

Chapter 15: JVM, JDK, JRE & Memory

The JVM loads classes, verifies bytecode, manages runtime memory and executes code. Modern JVMs may use JIT compilation to optimize frequently executed code.

Area	Conceptual role
Stack	Per-thread method-call frames and local execution state
Heap	Objects and arrays are generally allocated here
Class/runtime metadata	Information associated with loaded classes; exact organization depends on JVM implementation

Garbage Collection

GC automatically reclaims heap objects that are no longer reachable. Java normally does not require explicit freeing of ordinary objects.

■ Avoid the oversimplification 'GC deletes variables'. GC primarily concerns unreachable objects.

Chapter 16: equals(), hashCode(), final & Immutability

equals() defines logical equality when overridden. hashCode() returns a hash value consistent with that equality.

- If a.equals(b) is true, their hash codes must be equal.
- Equal hash codes do not guarantee objects are equal.
- Hash-based collections rely on this contract.

```
class Student {
    int id;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student s = (Student) o;
        return id == s.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

final

- final variable → cannot be assigned again after initialization.
- final method → cannot be overridden.
- final class → cannot be subclassed.

Immutability means an object's observable state cannot change after construction. String is a standard example.

Chapter 17: Debugging, Complexity & Problem Solving

A practical debugging cycle is: reproduce → isolate → inspect → form hypothesis → test → fix → retest.

Pattern	Typical complexity
One loop over n items	$O(n)$
Two full nested loops	$O(n^2)$
Binary search	$O(\log n)$
HashMap average lookup	$O(1)$ average; implementation/runtime details matter

Maximum element example

```
int max = arr[0];
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) max = arr[i];
}
```

Logic: keep the largest value seen so far. Time $O(n)$, extra space $O(1)$.

Edge cases

- Empty input.
- One element.
- Duplicates.
- Negative values.
- Already sorted / reverse sorted.
- Integer overflow.
- Very large input.
- Null references where applicable.

Chapter 18: Final Revision + Important DSA Questions

Final Revision Sheet

Concept	Quick meaning
Variable	Named storage associated with a type
Method	Reusable block of behavior
Class/Object	Blueprint / instance
Encapsulation	Controlled access to state
Inheritance	Subclass extends superclass
Polymorphism	Common type with different runtime behavior
Abstraction	Expose essentials, hide details
Exception	Abnormal condition handled by exception mechanisms
ArrayList	Resizable indexed list
HashMap	Key-value structure
Stream	Data-processing pipeline
JVM	Runtime that executes bytecode

■ Most Important Java Concepts

- Syntax and control flow
- Methods and recursion
- Arrays and Strings
- Classes and constructors
- Four OOP pillars
- Exceptions
- Collections
- Generics
- Streams
- JVM basics
- equals/hashCode

■ Important DSA Questions

Arrays & Strings

- Find max/min
- Reverse array
- Check sorted array
- Move zeroes
- Remove duplicates from sorted array
- Two Sum
- Majority Element

- Best Time to Buy/Sell Stock
- Maximum Subarray
- Product of Array Except Self
- Palindrome
- Longest Substring Without Repeating Characters
- Valid Anagram / Group Anagrams

Searching & Sorting

- Linear Search
- Binary Search
- First/last occurrence
- Search in rotated sorted array
- Merge Sort
- Quick Sort
- Kth largest
- Count inversions

Two Pointers / Sliding Window / Prefix Sum

- Two Sum sorted
- Container With Most Water
- 3Sum
- Fixed-size maximum sum window
- Longest valid window/subarray
- Subarray sum using prefix sums

Linked List

- Reverse Linked List
- Middle of list
- Detect cycle
- Merge two sorted lists
- Remove nth node from end
- Intersection of two lists

Stack / Queue

- Valid Parentheses
- Next Greater Element
- Min Stack
- Queue using stacks
- Sliding Window Maximum
- Largest Rectangle in Histogram

Trees / BST

- Preorder/inorder/postorder

- Level order
- Height
- Balanced tree
- Lowest Common Ancestor
- Validate BST
- Kth smallest in BST

Heap / Graph / DP

- Kth largest/smallest
- Top K frequent
- Merge K sorted lists
- BFS/DFS
- Number of Islands
- Connected Components
- Cycle Detection
- Topological Sort
- Dijkstra
- Climbing Stairs
- House Robber
- 0/1 Knapsack
- LCS
- Coin Change

Interview & Viva

- Why is binary search $O(\log n)$?
- Array vs ArrayList?
- HashMap vs HashSet?
- Overloading vs overriding?
- Why override hashCode with equals?
- Checked vs unchecked exception?
- start() vs run()?
- What is runtime polymorphism?
- What is Big-O?
- When can two pointers/sliding window help?

Last-Minute Revision

Java: syntax → types → conditions/loops → methods → arrays/strings → classes/objects → OOP → exceptions → collections → generics → streams → JVM.

DSA: complexity → arrays/strings → hashing → two pointers/sliding window → sorting/binary search → stack/queue → linked list → trees → heap → graphs → recursion/backtracking → DP.

■■ Final rule: understand the pattern before memorizing code. Trace small examples by hand, then implement the solution yourself.